CHAPTER 1

INTRODUCTION

1.1INTRODUCTION

The growing reliance on digital solutions has transformed how people access services in their daily lives, leading to a surge in demand for platforms that can efficiently connect users with skilled professionals. Traditional methods of finding service providers often involve time-consuming searches, lack of transparency, and limited trust. To address these issues, the Workers App project introduces a comprehensive and user-friendly platform designed to bridge the gap between users seeking services and service providers offering them. By leveraging both a web portal and a mobile application, the platform ensures that each user type—administrators, service providers, and customers—has access to features tailored specifically to their needs.

The system is structured around three main user roles. Administrators use the web portal to manage user accounts, monitor job activity, resolve complaints, and enforce service quality. Service providers also access the web portal to create profiles, define their services, update availability, and interact with customers through a built-in chat system. End-users, on the other hand, access the platform via a mobile application that allows them to search for nearby service providers, view ratings and reviews, send job requests, communicate with workers, and make secure payments. This role-based architecture ensures an efficient and personalized experience for all participants.

One of the standout features of the Workers App is its intelligent worker identification system, powered by Natural Language Processing (NLP). By analyzing user reviews and ratings using sentiment analysis and keyword recognition, the system highlights top-rated workers, helping users make informed decisions based on real feedback. This not only increases user trust but also motivates service providers to maintain high standards. Alongside this, the platform includes essential features like real-time notifications, a robust complaint mechanism, and an intuitive interface, all of which contribute to a seamless, secure, and engaging service experience.

1.2SCOPE

The scope of the **Connecting Workers** project includes the design, development, and deployment of a full-featured digital platform that facilitates the discovery, selection, and hiring of service providers. The system supports role-specific access for three user types—Admin, Service Provider, and User—each with customized functionality. The web portal provides tools for administrators to monitor system performance and for service providers to manage their offerings. The mobile application enables end-users to interact with the platform through intuitive search, booking, payment, and feedback mechanisms. Core features include real-time notifications, secure payment integration, complaint filing, and a robust review and rating system. Additionally, the inclusion of NLP for intelligent review analysis and provider ranking sets this platform apart, ensuring a smarter, more reliable user experience. The project aims to offer a secure, user-friendly, and efficient environment for service exchange, promoting better access to services and greater visibility for local workers.

1.3 OVERVIEW OF THE PROJECT

The **Connecting Workers** is a digital platform designed to connect people who offer services (like plumbers, electricians, cleaners, etc.) with users who need those services. It includes a website for administrators and service providers, and a mobile app for users. Administrators can manage accounts, job listings, and complaints. Service providers can create profiles, list their services, set their availability, and chat with users. Users can search for nearby workers, view their profiles and ratings, send service requests, and make secure payments through the app. The platform also uses Natural Language Processing (NLP) to analyze user reviews and highlight the best-rated workers, helping users make better choices. Overall, it aims to make finding and hiring trusted local workers fast, easy, and secure.

Module Details

1. Admin Module

- Secure login for administrators.
- Approve or reject worker registrations.
- Access job and worker activity details.
- View user profiles and their activity.
- Respond to complaints.
- Generate platform usage reports and statistics.

2.Worker Module

- Register and login securely.
- Update service information, availability, and location.
- Manage job details (images, rates, work hours).
- Accept or reject service requests.
- Communicate with users via chat.
- View feedback and performance insights

3. User Module

- Secure registration and login.
- Search services by keyword, location, or rating.
- Send and track service requests.
- Chat directly with workers.
- Rate services and file/view complaints

CHAPTER 2
SYSTEM ANALYSIS

System analysis is a logical process; it is an important activity that takes place when a new system is being built. The objective of this phase is not actually to solve the problem but to determine what must be done to solve the problem. It is the central intact of system development and it includes gathering and interpreting facts, diagnosing and using this information to recommend improvements to the system. System analysis gives the structure and functioning of the system and it specifies what the system should do. It is helpful to understand the problem and emphasize what is needed from the system.

Before designing an application which will help its users, it is important that the way it currently operates should be clearly identified. The process of system investigation includes several methods of gathering the required information. It is important that the approach is appropriate to the application under consideration and the user of the system. System analysis should be creative and imaginative in producing new solutions to meet the user requirements. In short the system is analysed by gathering various information such as system functionality, problems encountered, requirement by the proposed system, users, their tasks and responsibilities. Here in the system analysis phase, system is viewed as a whole and inputs to the system are in defined, and also the output from the system is traced through the various data collected.

A detailed study of these processes must be made by the various techniques like interviews, questionnaires, etc. The data collected by these sources must be scrutinized to arrive to a conclusion. The conclusion is an understanding of how the system functions. This system is called the existing system. Now, the existing system is subject to close study and the problem areas are identified. The designer now functions on a problem solver and tires to sort out the difficulties that weighed with the existing faces. The solutions a given on a proposal. The proposal is then weighed with the existing system analytically and the best one is selected. The

proposal is presented to the user for any endorsement by the user. The proposal is reviewed on user request and suitable changes that made. This is a loop that ends as soon as the user in satisfied with the proposal.

2.1 EXISTING SYSTEM

The existing system for connecting users and service providers is fragmented, lacking integration and efficient communication, often relying on third-party apps or manual methods. In contrast, the proposed system offers a unified platform with role-specific features, real-time communication, secure payments, and intelligent recommendations, ensuring a more efficient and seamless experience for users, service providers, and administrators**.**

1. Platform Integration

The existing system lacks a unified platform, with users and service providers often relying on separate third-party apps or manual methods. This results in inefficiency and fragmentation in managing services and user interactions.

2. User Roles

Most current systems do not provide clear distinctions between the roles of admins, service providers, and users. As a result, access control and functionality are not well-defined, leading to confusion and inconsistent management.

3. Admin Capabilities

Administrators in existing systems have minimal control over user management and service listings. The lack of tools to monitor service quality or handle complaints leaves room for unregulated service offerings**.**

4. Service Provider Management

Service providers often do not have an organized way to manage their profiles, availability, or service offerings. Without a centralized system, it is harder for workers to engage with potential customers effectively.

5. Communication

Existing systems do not provide built-in communication channels, and users and service

providers rely on phone calls or third-party messaging apps. This lack of real-time communication makes coordination between users and workers less efficient.

2.2 PROPOSED SYSTEM

The proposed **Connecting Workers** is an integrated platform designed to bridge the gap between service providers (workers) and users seeking services, accessible through both web and mobile interfaces. It is structured to support three distinct roles—**Administrator**, **Service Provider**, and **User**—each with specific features tailored to their needs.

- **Integrated Platform**: The proposed system offers a unified web and mobile platform with role-based access for Admins, Service Providers, and Users. This ensures smooth functionality across all devices and user types.
- **Admin Features**: Administrators can manage user accounts, monitor job listings, resolve complaints, and maintain service quality. This helps keep the platform secure, reliable, and well-organized.
- **Service Provider Portal**: Service providers can register, update their profiles, set availability, and communicate with users through an inbuilt chat system. This enables better scheduling and direct interaction with clients.
- **User Mobile App**: Users can browse nearby workers, view their profiles, check availability, send job requests, and make secure payments. The app is designed to make hiring local services quick and easy.
- **Secure Payment Gateway**: A built-in payment system allows users to pay securely within the app. This avoids cash handling and ensures smooth transactions for both users and providers.
- **Intelligent Recommendation System**: The platform uses Natural Language Processing (NLP) to analyse reviews and highlight top-rated service providers. This builds user trust and helps in making informed decisions.

- **Real-Time Notifications**: Users and service providers receive instant updates on job status, messages, and other activities. This improves communication and responsiveness.

2.2.1 ADVANTAGES OF PROPOSED SYSTEM

The "CONNECTING WORKERS" project presents a comprehensive platform that offers numerous advantages for all stakeholders involved—administrators, service providers, and users. Here are some of the key benefits:

Advantages of the "CONNECTING WORKERS" System:

1. Enhanced Accessibility:
   • The integrated web and mobile platforms ensure that all users can access services anytime and anywhere, increasing convenience for both service providers and users.

2. Role-Specific Functionality:
   • Tailored features for administrators, service providers, and users facilitate efficient interactions and streamline processes specific to each role, enhancing user experience.

3. Efficient Service Matching:
   • The intelligent worker identification system utilizing NLP allows for better matching between users and service providers based on reviews and ratings, leading to higher satisfaction rates.

4. Improved Trust and Transparency:
   • The robust rating system and sentiment analysis foster trust among users by highlighting top-rated service providers, encouraging quality service delivery.

5. Real-Time Communication:
   • The built-in chat system enables seamless communication between users and service providers, allowing for quick clarifications and fostering a collaborative environment.

2.3 FEASIBILITY STUDY

An initial investigation culminates in a proposal that determine whether an ultimate system is feasible. When a proposed system is made and approved, it initiates a feasibility study. The purpose of feasibility study is to identify various candidate systems and evaluates whether they are feasible by considering technical, economical, operational and legal feasibility and to recommend the best candidate system.

Feasibility study is conducted to test whether the system is beneficial to the organization. Feasibility study of this project includes the following.

2.3.1 Technical Feasibility

Technical feasibility study deals with the hardware as well as the software requirements. The scope was whether the work for the project is done with the current equipment and the existing software technology. The outcome was found to be positive.

The system “Connecting Workers” is said to be technically feasible because the software and hardware required to develop this system is already available. For this system we use **HTML** as the front end, and **Python** as the back end.

2.3.2 Operational Feasibility

The proposed project would be beneficial to organization satisfying the objectives when developed and installed. One of the main problems faced during the development of a new system is getting the acceptance from the user. There is support from the management of organization towards the development of the project. All the operational aspects are considered carefully. This the project is operationally feasible.

2.3.3 Economic Feasibility

The developing system must be justified by cost and benefit a criterion to ensure that effort taken on the project gives the best return at the earliest. One of the factors, which affect the development of a new system, is the cost it would require. Since the system developed is a part

of a project work, there is no manual cost to spend for the proposed system. Also, all the resources are already available, giving an indication that the system is economically possible for development.

The system “Connecting Workers” is said to be economically feasible because the hardware and software required to develop the system is already available. No additional requirements are needed, so there is no need to purchase any tools from outside. So it is a cost effective system. Its maintenance cost is also very cheap. This project is done as part of our course. Hence this is economically feasible.

2.3.4 Legal Feasibility

Legal feasibility is the determination of any infringement, violation, or liability that could result from the development of the system. Legal feasibility environment passes abroad range of concerns that include contract and liability. The proposed project is also a legally feasible one.

CHAPTER 3

SYSTEM SPECIFICATION

3.1 SYSTEM SPECIFICATIONS

3.1.1 HARDWARE SPECIFICATIONS

The selection of hardware is very important in the existence and proper working of any software. Then selection hardware, the size and capacity requirements are also important.

- Processor - Intel x86
- Speed - 1.1 GHz
- RAM - 700 MB (min)
- Hard Disk - 150 MB
- Key Board - Standard Windows Keyboard
- Mouse - Two or Three Button Mouse
- Monitor - SVG

3.1.2 SOFTWARE SPECIFICATIONS

One of the most difficult tasks is selecting software for the system, once the system requirements is found out then we have to determine whether a particular software package fits for those system requirements. The application requirement:

- Operating System - Windows 7 or Above
- Web Server - Wamp Server
- Backend - Python
- Database - MySql
- Platform use - JetBrains, PyCharm, Android Studio
- Web Browser - Google Chrome, Fire fox, Microsoft Edge
- Front End - HTML, CSS, JAVASCRIPT
- Frame work - Flutter

3.2 SOFTWARE TECHNOLOGY OVERVIEW

3.2.1 Python

Python is a dynamic, high level, free open source and interpreted programming language. It supports object-oriented programming as well as procedural oriented programming.

In Python, we don't need to declare the type of variable because it is a dynamic typed language. For example, x=10, here x can be anything such as String, into etc.

Features in Python

There are many features in Python, some of which are discussed below –

1.Easy to code

Python is high level programming language. Python is very easy to learn language as compared to other language like c, c#, java script, java etc. It is very easy to code in python language and anybody can learn python basic in few hours or days. It is also developer-friendly language

2.Free and Open Source:

Python language is freely available at official website and you can download it. Since, it is open-source; this means that source code is also available to the public. So you can download it as, use it as well as share it.

3.Object-Oriented Language:

One of the key features of python is Object-Oriented programming. Python supports object oriented language and concepts of classes, objects encapsulation etc.

4.GUI Programming Support:

Graphical Users interfaces can be made using a module such as PyQt5, PyQt4, python or Tk in python. PyQt5 is the most popular option for creating graphical apps with Python.

5.High-Level Language:

Python is a high-level language. When we write programs in python, we do not need to remember the system architecture, nor do we need to manage the memory.

6.Extensible feature:

Python is an Extensible language. We can write some python code into c or c++ language and also we can compile that code in c/c++ language.

7.Python is Portable language:

Python language is also a portable language. For example, if we have python code for windows and if we want to run this code on other platform such as Linux, Unix and Mac then we do not need to change it, we can run this code on any platform.

8.Python is integrated language:

Python is also an integrated language because we can easily integrated python with Other language like c, c++ etc.

9.Interpreted Language:

Python is an Interpreted Language. Because python code is executed line by line at a time. Like other language c, c++, java etc. there is no need to compile python code this makes it easier to debug our code. The source code of python is converted into an immediate form called byte code.

10.Large Standard Library:

Python has a large standard library which provides rich set of module and functions so you do not have to write your own code for every single thing. There are many libraries present in python for such as regular expressions, unit-testing, web browsers etc.

11. Dynamically Typed Language

Python is dynamically-typed language. That means the type (for example- int, double, long etc.) for a variable is decided at run time not in advance. Because of this feature we don't need to specify the type of variable.

3.2.2Flutter

Flutter is a popular open-source UI software development toolkit created by Google, primarily used for building natively compiled applications for mobile, web, and desktop from a single

codebase. It uses the Dart programming language and provides a rich set of pre-designed widgets that make it easy to create visually appealing and responsive user interfaces.

Key Features of Flutter:

1. Single Codebase: Write once, run anywhere. You can develop applications for iOS, Android, web, and desktop using the same codebase.
2. Hot Reload: Flutter's hot reload feature allows developers to see the results of their changes in real-time without restarting the application. This speeds up the development process significantly.

3. Rich Widgets: Flutter provides a wide range of customizable widgets that follow the Material Design and Cupertino (iOS) design guidelines, allowing developers to create beautiful interfaces easily.

4. Performance: Flutter apps are compiled to native ARM code, which helps achieve high performance on both Android and iOS devices.

5. Strong Community Support: Being backed by Google and having a large community means there are plenty of resources, packages, and plugins available to extend functionality.

6. Integration with Other Technologies: Flutter can easily integrate with various backend services, databases, and APIs, making it suitable for a wide range of applications.

Use Cases for Flutter:

* Mobile Applications: Ideal for developing cross-platform mobile applications that require a native look and feel.

* Web Applications: Flutter can also be used to create responsive web applications.

* Desktop Applications: With Flutter's support for desktop platforms, you can build applications for Windows, macOS, and Linux. processing HTTP requests with a web templating system (View), and a regular-expression- based URL dispatcher (Controller).

Java Script

Java script is a scripting language that can be used to create client-side scripts and server side scripts. Client-side scripts are executed in the browser while server-side scripts are executed on a server. We can insert JavaScript statements directly into an HTML page. When the page is displayed in the browser, the JavaScript statements are interpreted and executed by the browser. JavaScript statements can recognize and respond to user events such as mouse clicks or system generated events and so on. So you can change the content and position of the elements on the page dynamically, in response to user interaction.

When the client requests an HTML page that includes a client-side Script, the server forwards the full content of the HTML document- the JavaScript statements and the HTML content. When the browser receives the document, it executes the HTML and JavaScript statements without any interaction with the server while both client-side JavaScript and server- side JavaScript have the same core language; each also has additional features relevant to the environment. That is, client side JavaScript includes predefined objects that can be used only in the browser. Server-side JavaScript contain predefined objects that can be used in server-side application.

Hyper Text Mark Up Language

An HTML file is a text file containing small mark-up tags. These tags tell the web browser how to display the page. An HTML file must have an htm or html file extension. An HTML file can

be created by using a simple text editor. HTML documents are text files made up of HTML elements e.g.: <html>, <body>. HTML elements are defined using HTML tags.

HTML tags are used to mark-up HTML elements. The two characters surround HTML tags <and>. The surrounding characters are called angle brackets. HTML tags normally come in pairs like and . The first tag in a pair is the start tag: the second tag is the end tag. The text between the start and the end tag is the element content. HTML tags are not case sensitive; means the same as <B>.

3.2.2 MySQL

MySQL is an open-source relational database management system (RDBMS).Its name is a combination of "My", the name of co-founder Michael Widenius's daughter My and "SQL", the acronym for Structured Query Language. A relational database organizes data into one or more data tables in which data may be related to each other; these relations help structure the data. SQL is a language programmers use to create, modify and extract data from the relational database, as well as control user access to the database.

In addition to relational databases and SQL, an RDBMS like MySQL works with an operating system to implement a relational database in a computer's storage system, manages users, allows for network access and facilitates testing database integrity and creation of backups.

MySQL is the world's most popular open source database software, with over 100 million copies of its software downloaded or distributed throughout its history. With its superior speed, reliability, and ease of use, MySQL has become the preferred choice for Web, Web 2.0, SaaS, ISV, Telecom companies and forward-thinking corporate IT Managers because it eliminates the major problems associated with downtime, maintenance and administration for modern, online applications.

Many of the world's largest and fastest-growing organizations use MySQL to save time and money powering their high-volume systems, and packaged software – including industry leaders such as Yahoo Alcatel-Lucent, Google, Nokia, YouTube, Wikipedia, and Booking.com.

The flagship MySQL offering is MySQL Enterprise, a comprehensive set of production-tested software, proactive monitoring tools, and premium support services available in an affordable annual subscription.

MySQL is a key part of LAMP (Linux, Apache, MySQL, PHP Perl / Python), the fast-growing open source enterprise software stack. More a companies are using LAMP as an alternative to expensive proprietary software stacks because of its lower cost and freedom from platform lock-in.

The MySQL database is owned, developed and supported by Sun Microsystems, one of the world's largest contributors to open-source software. MySQL was originally founded and developed in Sweden by two Swedes and a Finn David Axmark, Allan Larsson and Michael "Monty" Widenius, who had worked together since the 1980's. The best and the most-used database in the world for online applications

- Available and affordable for all
- Easy to use
- Continuously improved while remaining fast, secure and reliable
- Fun to use and improve Free from bugs

Wamp Web Server

Wamp Server is a robust and widely used software stack that provides a comprehensive platform for web development, testing, and deployment. The acronym WAMP stands for Windows, Apache, MySQL, and PHP/Python, representing the four core components that make up this powerful suite. It combines Apache, the widely used web server software known for its robustness and flexibility, with MySQL, a powerful relational database management system, and PHP, a scripting language favoured for server-side web development. Wamp Server provides an easy-to-install package that allows developers to set up a local web server environment quickly to develop and test websites or web applications before deploying them to a live server. It's popular among developers due to its simplicity and all-in-one nature for web development on Windows platforms. It is a comprehensive software stack primarily used for local web development on Windows systems. & DEVELOPMENT

CHAPTER 4

SYSTEM DESIGN & DEVELOPMENT

4.1 INFRASTRUCTURE DESIGN

System design provides an understanding of the procedure details, necessary for implementing the system recommended in the feasibility study. Basically it is all about the creation of a new system. This is crucial phase since it decides the quality of the system and has a major impact on the testing and implementation phase. Design is the second phase in the system development life cycle software design is the first of the three technical activities in the software development process such as design, code writing and testing.

During this phase, the analyst schedules design activities, works with the user to determine the various data inputs to the system, plans how data willow though the system, designs required outputs and writes program specifications. Again the analyst's activities focus on solving a user's problem in logical terms.

During this second step, analysts employ s variety of tools such as data flow

diagrams, entity- relationship diagrams, data dictionaries and Gantt chart. The system's design converts the theoretical solution introduced by the feasibility study into a logical reality.

During design the analyst:

- Draws a model of the new system, using data flow and entity-relationship diagrams
- Develop methods for collecting and inputting data
- Defines the detailed data requirements with a data dictionary
- Writes program specifications
- Specifies control techniques for the system's outputs, databases and inputs.
- Identifies and orders any hardware or software that the system will need.

4.2 INPUT DESIGN

Input Design is the process of converting a user-oriented description of the inputs to a computer based business system into a programmer-oriented specification. The aim of making input design is to make the data entry as easy, logical and free from errors as possible. An input format should be logical and easy to understand. In the design, the user oriented inputs are converted into computer recognizable format.

The collection of data is most expensive part of the system in terms of the equipment used, time and no. of clients involved etc. in the input design data is accepted and it can be readily used for data processing or can be stored in a database for further use. The activities used for inputs are very user friendly. Different names are associated with each data entry activity item makes data entry an easy job. Each data entry contains separate buttons for submitting the form and proper validation checking is carried out and necessary message will be presented to the user in case of improper data entry. The proposed system satisfies the following input design objectives

- A cost effective method of input
- The highest possible level of accuracy
- The input is acceptable to and understood by the user.

Input objectives are

i. **Controlling the amount of input:** Wherever user input is required, the number of key strokes is reduced by giving possible input values as default in that area. The viewer can select the answer in single click. The amount of information entered by the viewer using the virtual keypad is reduced to the maximum and the software is made very user friendly.

ii. **Avoiding Delay:** A processing delay resulting from data preparation or data entry operations is called a bottleneck. Such bottlenecks are avoided to the maximum. The only time the viewer has to wait is when the file is uploaded or downloaded. Progress

bar or progress meters are displayed to keep the user waiting and also to show the speed and amount of download.

iii. Avoiding Errors in Data: The rate at which errors occur depends on the quantity of data. Here the quantity of data is reduced to the lowest, and a text file is easily manageable .

iv. **Avoiding Extra steps:** The viewer can quit at any point of time. Even at the time of upload or download the viewer can quit. The viewer need not wait for any specific event to happen for quitting the process.

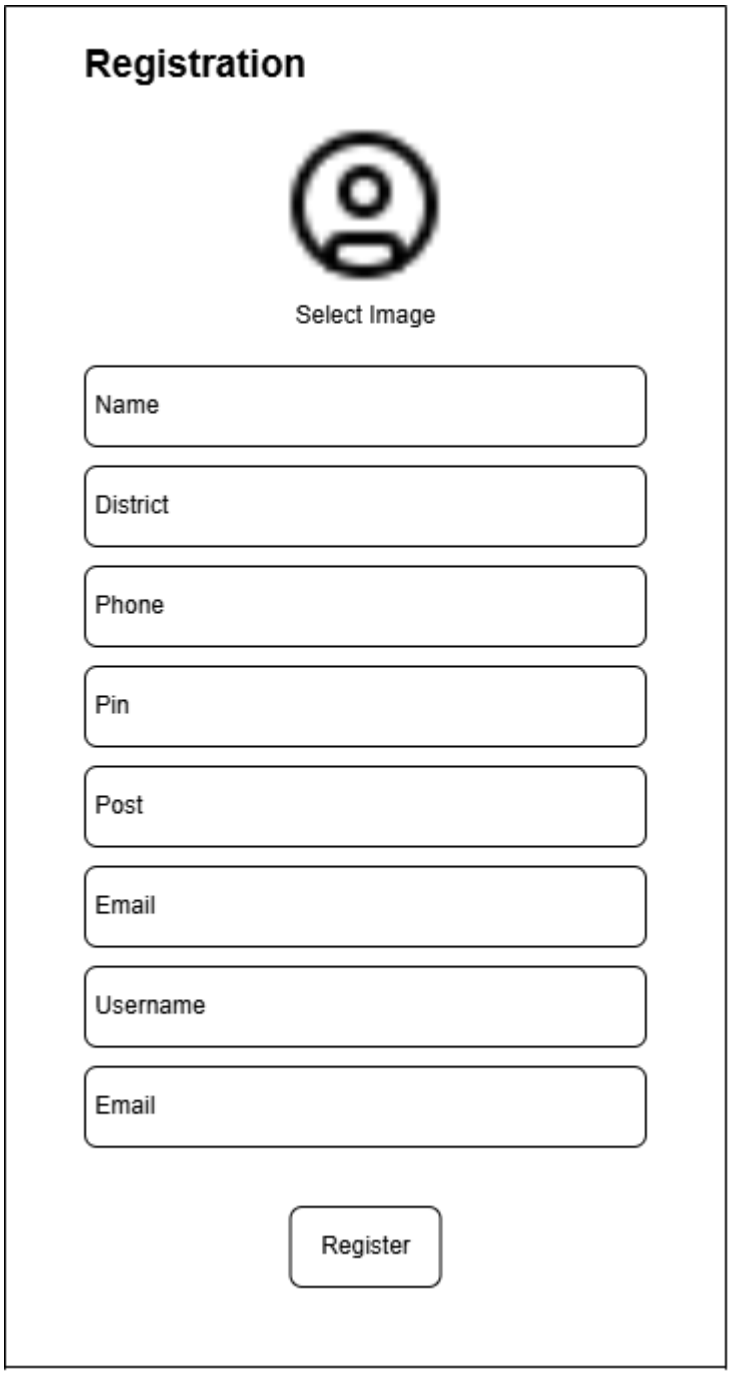
v. **Keeping the process Simple:** This implies that the system has all the measure to keep the errors out even if the user is giving wrong data. It handles the situation with grace and doesn't create much hype about the situation to the user.

4.3 OUTPUT DESIGN

It has been an ongoing activity from the beginning of the project. It includes the process of finalizing of the medium format and exact contents of each output to be produced by the proposed system. The primary objective in creating an output is accuracy and neatness.

Efficient output design should improve the system's relationship with the user and help in decision making. The system has got the capability to display standard screen layouts. These layouts should be designed around the output requirements and they must be designed with utmost care and details in the screen layouts must be simple, descriptive and clear to the user. While designing a system's output, the analyst must make several independent decisions. Every system produces some kind of reports. No matter what is the content of the report. The output design of this project is made with these objectives in mind.

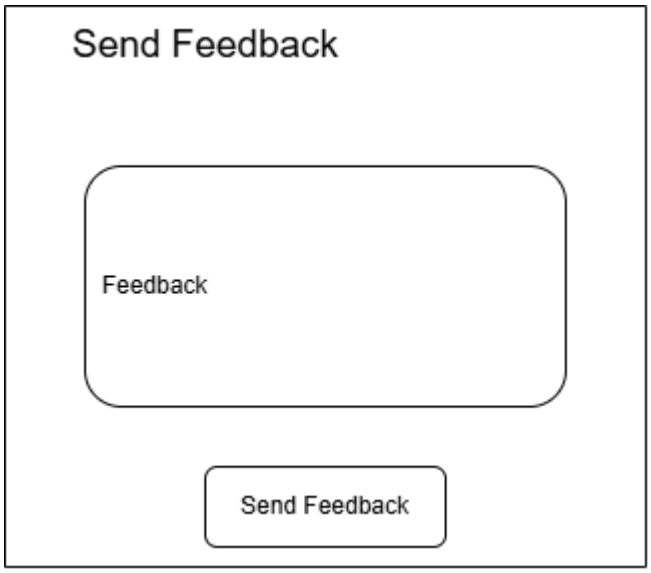
4.3.1 REGISTRATION FORM



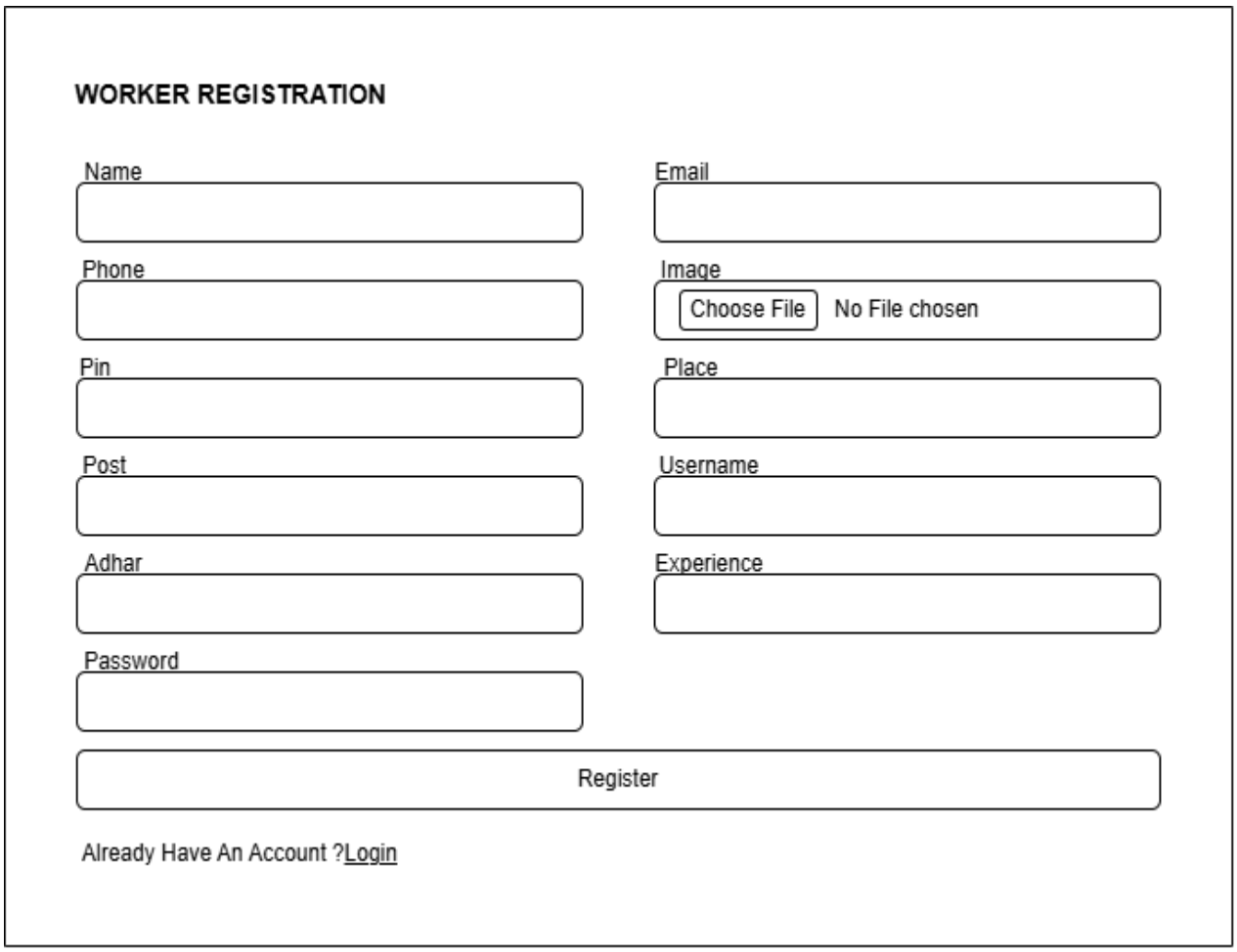
LOGIN FORM



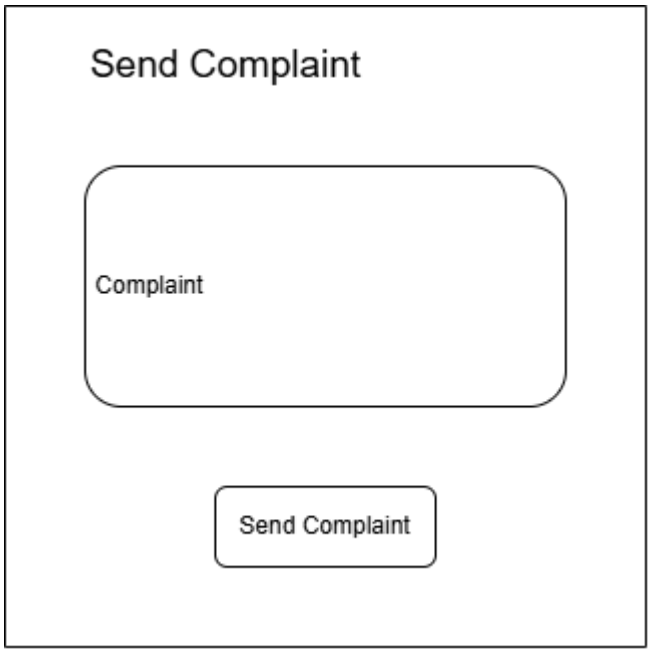
FEEDBACK FORM



WORKER FORM


WORKER REGISTRATION
Name
Email
Phone
Image
Choose File
No File chosen
Pin
Place
Post
Username
Adhar
Experience
Password
Register
Already Have An Account ?Login

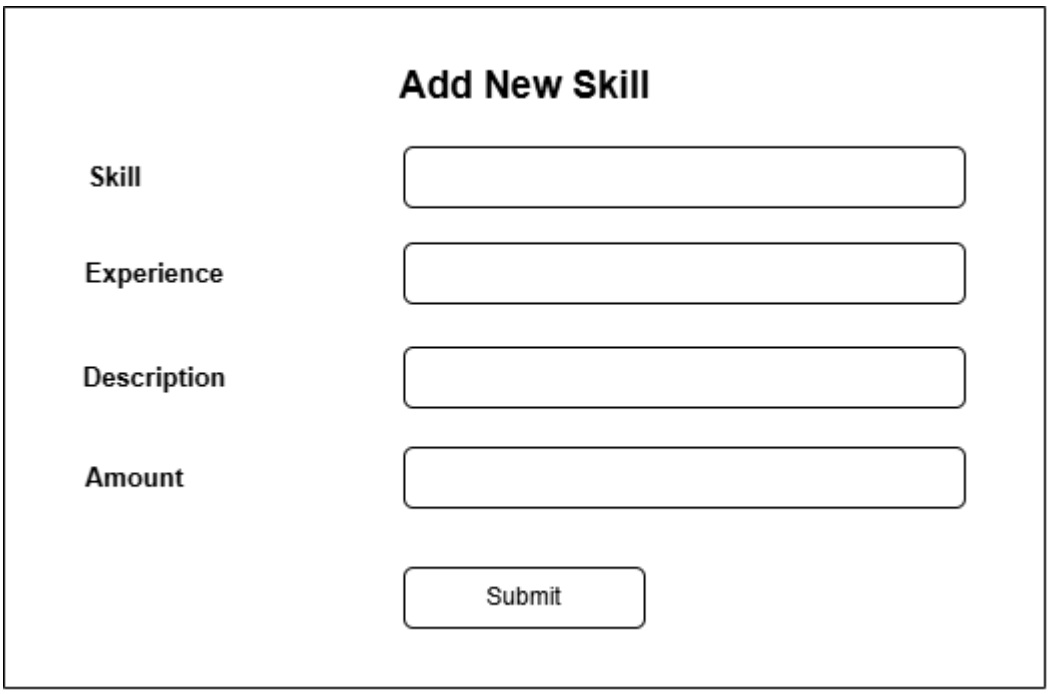
COMPLAINT FORM



WELCOME BACK LOG FORM



ADD SKILLS FORM



Output Types

- External Outputs, whose destination is outside the organization and is the main image of the organization,
- Internal Outputs, whose destination is within the organization and which require careful design because it is user's main interface with the Android mobile.
- Interactive Outputs, which involve the user in communicating directly with the Android mobile.

4.4 DATA FLOW DIAGRAM

A DFD, also known as a "bubble chart" has the purpose of clarifying system requirements and identifying major transformations that will become programs in system design. A DFD consists of a series of bubbles joined by lines. The bubbles represent data transformations and the lines represent data flow in the system.

A data flow diagram may be used to represent a system or software at any level of abstraction DFD's can be partitioned into levels that represent increasing information flow and functional details. A Data Flow Diagram (DFD) is a diagram that describes the flow of data and the processes that change or transform data throughout a system. It is a structured analysis and design tool that can be used for flowcharting in place of or in association with, information oriented and process oriented system flowcharts.

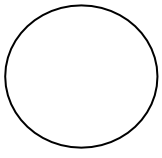
When analysts prepare the DFD, they specify the user needs at a level of detail that virtually determines the information flow into and out of the system and the required data resources. This network is constructed by using a set of symbols that do not imply a physical implementation. The DFD reviews the current physical system, prepares input and output specification, specifies the implementation plan etc.

Basic data flow diagram symbols are:

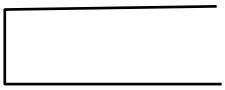


A “Rectangle” defines a source or destination

An “Arrow” identifies data flow. It is a pipeline through which information flows.



A “Circle” represents a process that transforms incoming data flow(s) into outgoing data flow(s).

 An “Open Rectangle” is a data store

Steps to Construct Data Flow Diagrams:

Three steps are commonly used to construct a DFD.

- The direction of flow is from top to bottom and from left to right.
- When a process is exploded in to lower level details they are numbered.
- Process should be named and numbered for easy reference each name should be representative of the process.

LEVEL 0 DFD (CONTEXT DIAGRAM)

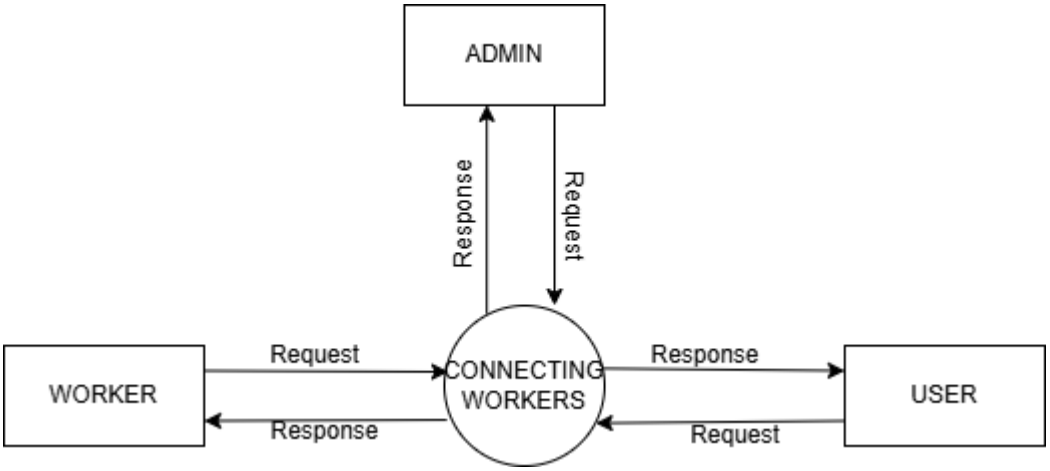


Fig: 4.4.1 Context Diagram

Level 1 Admin

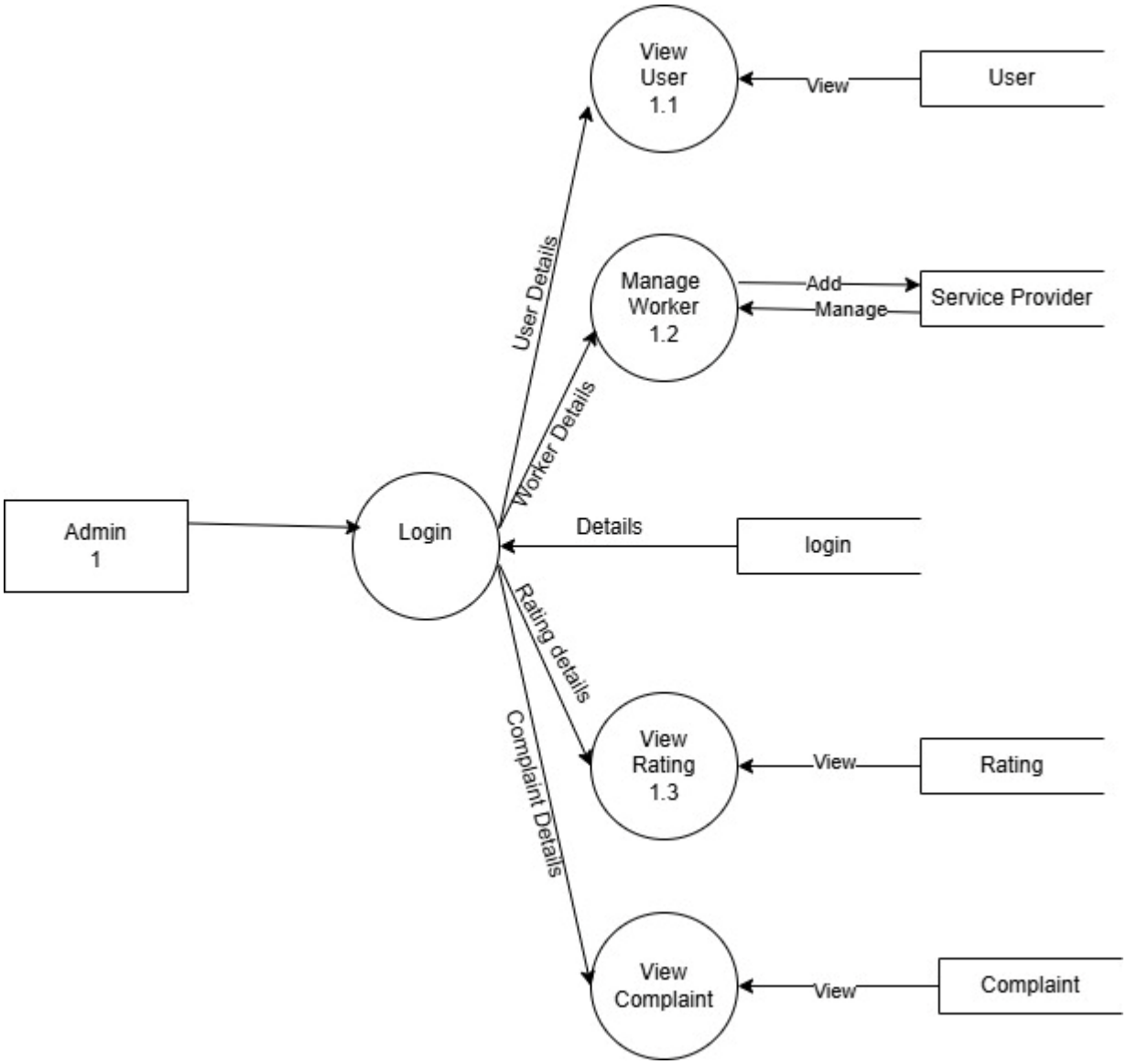


Fig: 4.4.2 Level 1 Admin

Level 1 User

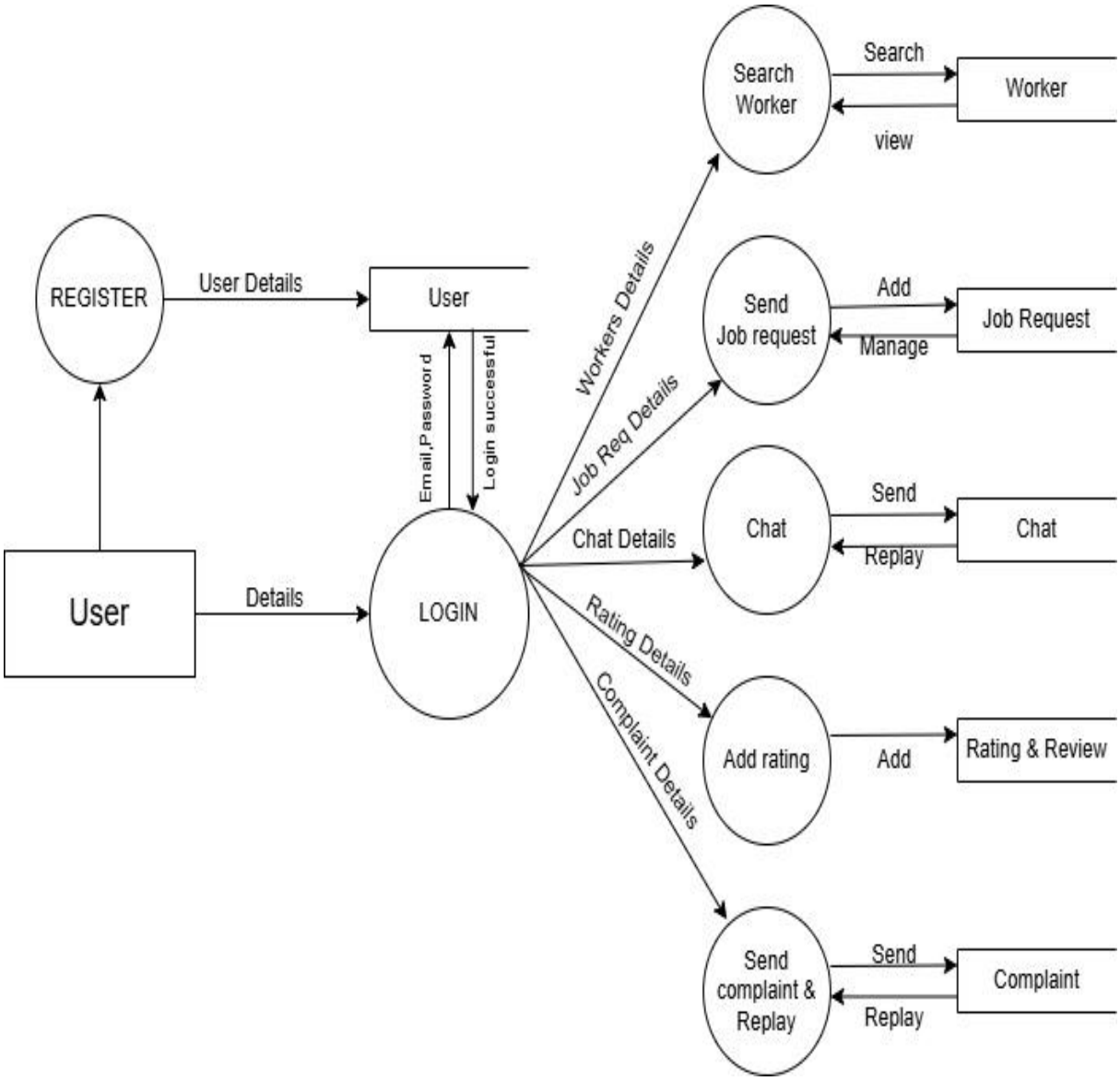


Fig: 4.4.3 Level 1 USER

Level 1 Worker

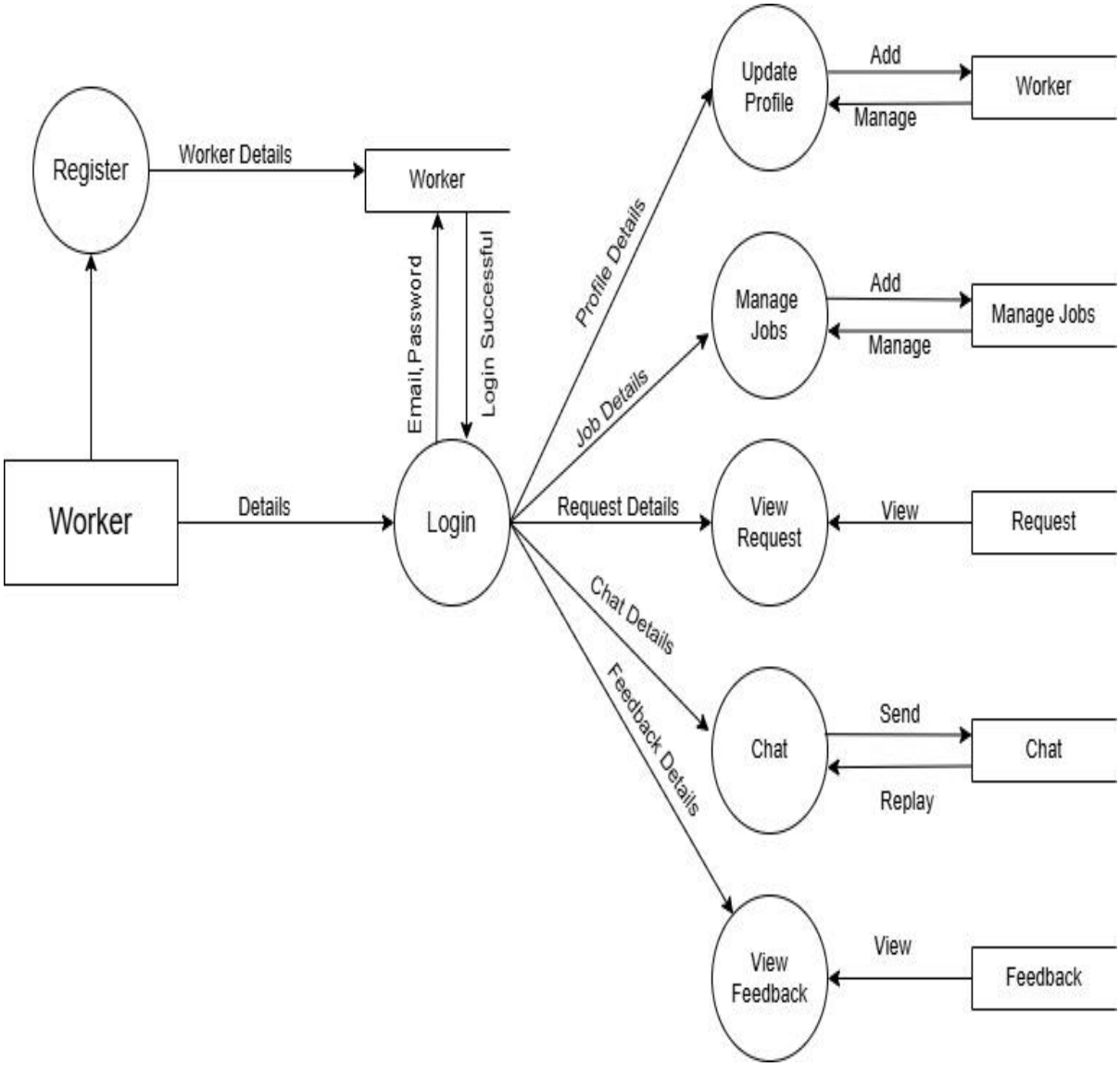


Fig: 4.4.4 Level 1 Worker

4.4 ANALYSIS TOOLS

System analysis is the process of collecting and interpreting facts, understanding problems and using this information to suggest improvements in the system. This will helps to understand the existing system and determine how computers make their operations more effective.

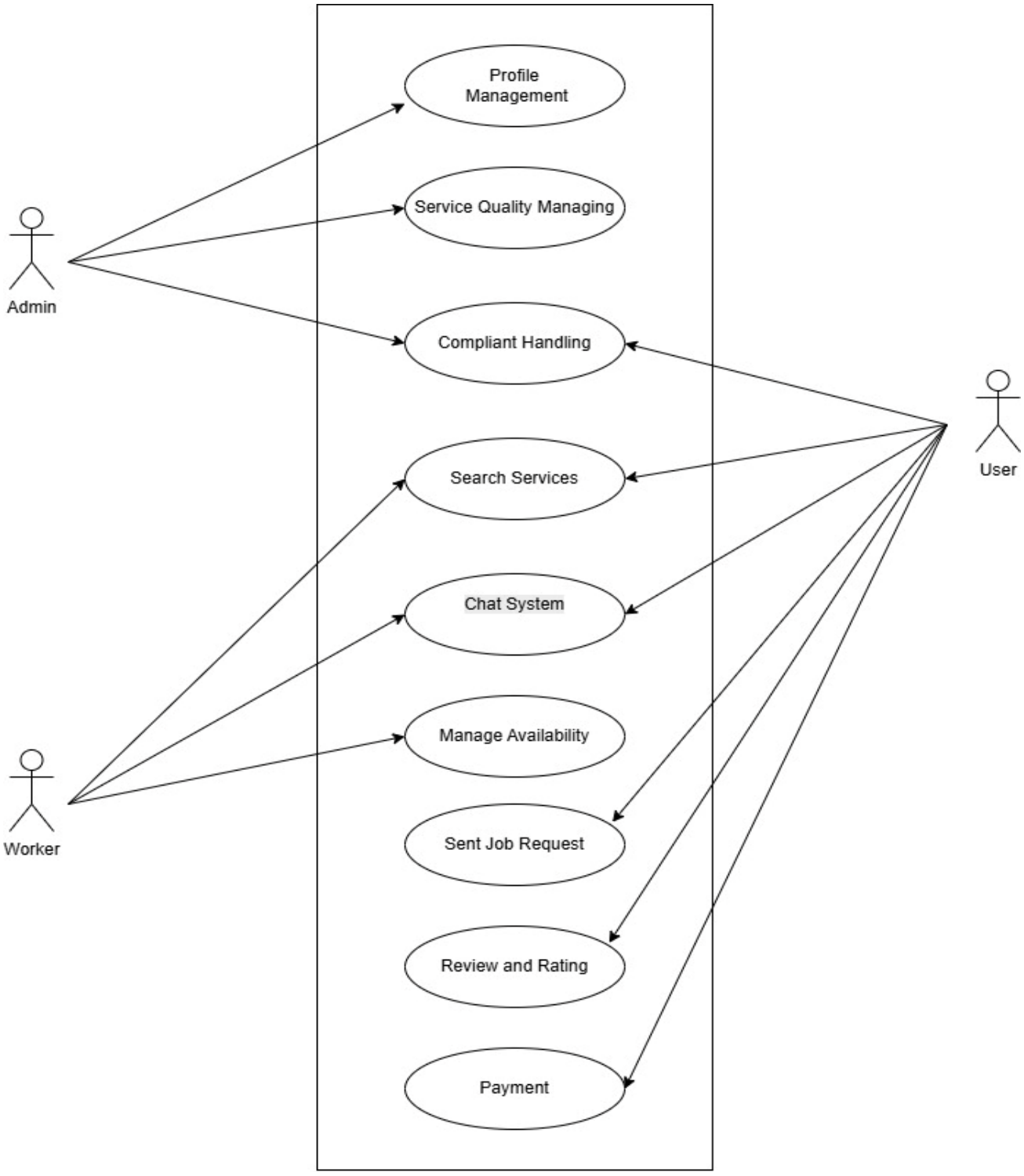
In software engineering, a use case diagram in the Unified Modelling Language (UML) is a type of behavioural diagram defined by and created from a Use-Case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals, and any dependencies between those use cases.

The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted. The use case diagram shows the position or context of the use case among other use cases. As an organizing mechanism, a set of consistent, coherent use cases promotes a useful picture of system behaviour, a common understanding between the customer/ owner/ user and the development team.

Use Case is a technique for capturing functional requirements of systems and systems-of-systems. Each use case provides one or more scenarios that convey how the system should interact with the users called actors to achieve a specific business goal or function. Use case actors may be end users or other systems. Use cases typically avoid technical jargon, preferring instead the language of the end user.

Use Case is a technique for capturing functional requirements of systems and Use Case is a technique for capturing functional requirements of systems and represented by ovals and the actors are represented by stick figures. The use case diagram of this application is given below.

4.3.1 USE CASE DIAGRAM



4.5 Database Design

After designing the input and output activities, the designer moves to concentrate on database design. How data are organized depends on the data and response requirements that determine hardware configurations. The database is organized to ensure that the system resources are not wasted. The objective of the database design is to provide the effective auxiliary storage and contribute to the overall efficiency to the computer program components of the proposed system. The database design translates the data models that were developed for the system users during the definition phase into the data structures supported by the chosen database technology.

The goals of database design are as follows:

1. A database should provide for the efficient storage, update and retrieval of data.
2. A database should be reliable; the stored data should have high priority to promote user trust in that data.
3. A database should be adaptable and scalable to new and unforeseen requirements and applications.

The techniques used to improve a data model in preparation for database design is called data analysis. Data analysis is a process that prepares a data model for implementation as a simple, non-redundant and adaptable database. The specific technique is called Normalization. Normalization is a technique that organizes the data attributes such that they are grouped to form stable, flexible and adaptive entities. The table involved in inspection process along with attributes, data types, constraints and brief description about the fields are stated in the below mentioned table.

4.5.1 Normalization

Normalization is the process of decomposing the attributes in an application, which results in a set of tables with very simple structure. The purpose of normalization is to make tables as simple as possible. Normalization is carried out in this application for the following reasons.

- To structure the data so that there is no operation of data, this helps in saving space.
- To permit simple retrieval of data in response to query and report request.
- To simplify the maintenance of the data through updates, insertions and deletions.
- To reduce the need to restructures or recognize data which new application requirements arise.

Primary key is assigned for this purpose. The primary key fields in the tables help to ease the search and improve efficiency. The proposed system is using second normal form as it is found most suitable. In second normal form each row must contain associated field that describes an attribute of the entry that the table describes.

Table 4.6.1 LOGIN TABLE

Fieldname	Type	Constraints	Description
id	int(11)	Primary key	Login ID
username	varchar(50)	Not Null	Username
password	varchar(100)	Not Null	Login Password
type	varchar(100)	Not Null	Login Type

Table 4.6.2 USER REGISTRATION

Fieldname	Type	Constraints	Description
id	int(11)	Primary key	User ID
name	varchar(50)	Not Null	Name of user
email	varchar(100)	Unique	Email ID
phone	varchar(12)	Not Null	Contact Number
place	varchar(100)	Not Null	Place of residence
post	varchar(100)	Not Null	Post Office
district	varchar(100)	Not Null	District
pin	varchar(6)	Not Null	Pincode
image	varchar(100)	Not Null	Profile Photo
LOGIN_ID	varchar(20)	Foreign key	Login ID

Table 4.6.4 WORKER REGISTRATION

Fieldname	Type	Constraints	Description
id	int(11)	Primary key	Worker ID
name	varchar(50)	Not Null	Name of Worker
email	varchar(100)	Unique	Email ID
phone	varchar(12)	Not Null	Contact Number
adharnumber	varchar(12)	Not Null	Adhar Number of worker
experience	varchar(10)	Not Null	Experience
place	varchar(100)	Not Null	Place of residence
post	varchar(100)	Not Null	Post Office
pin	varchar(6)	Not Null	Pincode
status	varchar(20)	Not Null	Status of Worker

image	varchar(100)	Not Null	Profile Photo

Table 4.6.3 NAME CHAT

Fieldname	Type	Constraints	Description
id	int(11)	Primary key	Chat ID
message	longtext	Not Null	Message
date	timestamp	Not Null	Date of Message send
from_id	int(20)	Not Null	ID of user or worker sending the message
to_id	int(20)	Not Null	ID of user or worker receiving the message

Table 4.6.5 COMPLAINT TABLE

Fieldname	Type	Constraints	Description
id	int(11)	Primary key	Complaint ID
Complaint	longtext	Not Null	Complaint
date	timestamp	Not Null	Date of camplaint Filed
reply	longtext	Not Null	Reply Message
user_id	int(11)	Foreign key	User ID

Table 4.6.11 FEEDBACK TABLE

Fieldname	Type	Constraints	Description
id	int(11)	Primary key	Feedback ID
feedback	longtext	Not Null	Feedback
date	timestamp	Not Null	Date of feedback issued
user_id	int(11)	Foreign key	User ID
worker_id	int(11)	Foreign key	Worker ID

Table 4.6.6 PAYMENT

Fieldname	Type	Constraints	Description
id	int(11)	Primary key	Payment ID
date	timestamp	Not Null	Date of payment done
amount	int(11)	Not Null	Transaction Amount
request_id	int(11)	Foreign key	REQUEST ID

Table 4.6.8 SKILL

Fieldname	Type	Constraints
id	int(11)	Primary key
skills	varchar(100)	Not Null
description	longtext	Not Null
experience	varchar(10)	Not Null

amount	int(11)	Not Null
worker_id	int(11)	Not Null

Table 4.6.7 STAFF TABLE

Fieldname	**Type**	**Constraints**	**Description**
id	int(11)	Primary key	Request ID
request	longtext	Not Null	Description of the service requested
status	varchar(20)	Not Null	Request Status
date	timestamp	Not Null	Date of request was made
workdate	timestamp	Not Null	Schedule date for the service
skill_id	int(11)	Foreign key	SKILL ID
User_id	int(11)	Foreign key	USER ID

4.6 E-R DIAGRAM

An entity-relationship (ER) diagram is specialized graphic that illustrates the interrelationship between entities in a database boxes are commonly used to represent entity Diamonds are normally used to represent relationships and ovals are used to represent attributes.

An entity is piece of data is shared between entities.

O Classifying Relationships

Relationships are classifieds by their degree, connectivity, cardinality, direction, type and existence.

O Degree of Relationships

The degree of a relationship is the number of entities associated with the relationship. The n-array relationship is there general from for degree n. Special cases are binary, ternary where the degree is 2 and 3 respectively.

O Connectivity and Cardinality

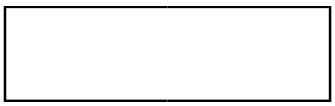
The connectivity of a relationship describes the mapping of associated entity instances in the relationship. The values of connectivity are "one" or "many" The cardinality of a relationship is the actual number of related occurrences for each of the two entities. The basic types of connectivity of relations are: One-to-one, one-to- many, many-to-many.

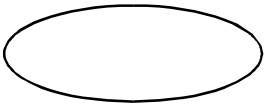
A one-to-one (1:1) is when at most one instance of an entity A is associated with one instance of entity B.

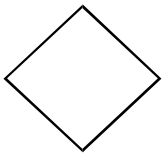
0-to-many (1: N) Is when for an instance of a entity A, there are zero, one or many instances of entity B, but for instance of the entity B, there is only one instance of the entity A.

A many-to-many (M: N) relationship, sometimes called non-specific, is when for one instance of entity A, there are zero, one or more instances of entity B and for one instance of entity B there are zero, one or many instances of entity A.

The symbols used ER Diagram is

Entity 

Attributes 

Relationships 

Lines

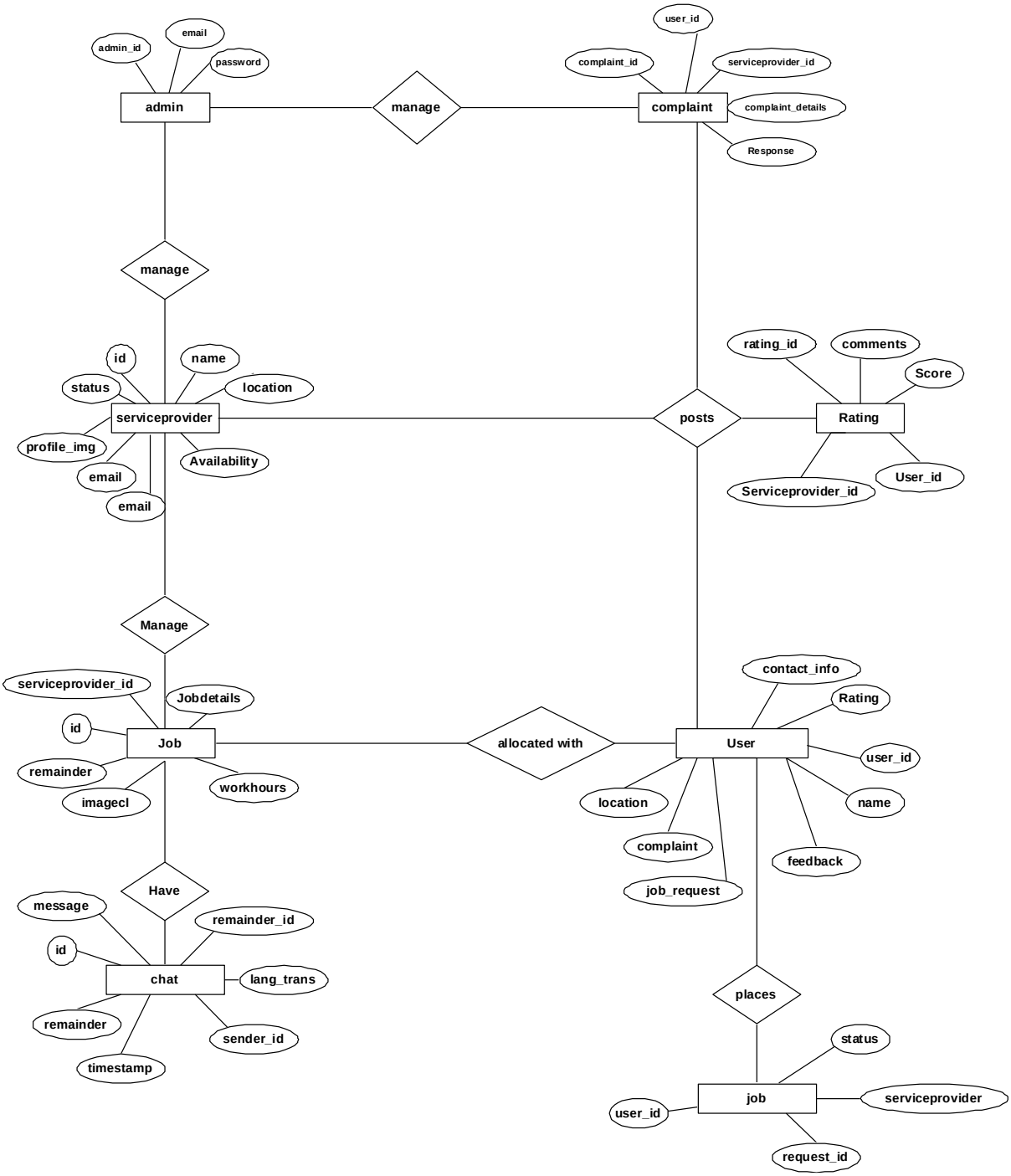


FIG: 4.7 ER Diagram

CHAPTER 5
SYSTEM TESTING

5.1 INTRODUCTION

System testing is actually a series of different testes whose primary purpose is to fully exercise the computer based system. Although each test has a different purpose, all work together to verify that the system elements have been properly integrated and perform all functions clearly. System testing makes logical assumptions that if all parts of the system are correct, the goal will be successfully achieved. Testing is the process of executing the program with the intent of finding errors. Testing cannot show the absence of defects, it can only show that software errors are present.

Testing on this project can be done in many ways, such as module testing, where every single program module is examined thoroughly, this project is also divided into many modules such service, user interface, viewing schedules and deleting schedules etc. Also, the whole unit will be tested as every data entered and searched will also be tested; all the runtime errors can be detected and corrected accordingly. There are also other types of testing such as integration testing; validation testing etc. the whole project is integrated so it has to be tested at each and every point.

Testing is a process of checking whether the developed system is working according to the original objectives and requirements. A test case is one that has a possibility of finding as yet undiscovered error. A successful test is one that uncovers as yet undiscovered error. The developed system is tested whether it works efficiently and whether it satisfies all the user requirements by taking a series of test cases.

5.1.1. Types of Testing

The software, which has been developed, has to be tested to prove its validity. Testing is considered to be least creative phase of the whole cycle of the system design. In the real sense

it is the phase, which helps to bring out the creativity of the phases. No system design is ever perfect. Errors occur due to communication problems, programmer's negligence or time constraints. All these must be eliminated before the system is ready for user acceptance testing. Different levels of testing are employed during different stages of the system development.

5.1.1.1 White Box Testing

By using this technique, it was tested that all the individual logical paths were executed at least once as logical decisions were tested on both their true and false sides. All the loops were tested with data in between the ranges and especially at the boundary values.

5.1.1.2 Black Box Testing

By the use of this technique, the missing functions were identified and placed in their positions. The errors in the interfaces were identified and corrected. This technique was used to identify the initialization and termination errors and correct them.

5.1.1.3 Unit Testing

In the lines of this strategy, all the individual functions and modules were put to the test independently. By following this strategy, all the errors in coding were identified and corrected. This method was applied in combination with the White Box and Black Box testing techniques to find the errors in each module.

5.1.1.4 Integration Testing

This testing strategy has two different approaches namely the top down approach, in which the integrations are carried out from the top level module to the bottom and bottom up approach in which the integration is carried out from the low level module to the top.

The modules were tested using the bottom up approaches by introducing stubs for the top-level functions. This test was used to identify the errors in the interfaces, the errors in passing the parameters between the functions and to correct them.

5.1.1.5 Validation Testing

Validation testing can be defined in many ways, but a simple definition that validation succeeds when the software in a manner that is reasonably expected by the customer. Software validation is achieved through a series of black box test that demonstrate conformability with requirements. After validation test have conducted, one of the two possible conditions exists.

- The function or performance characterized confirm the specification and are accepted.
- Deviation from specification is found and a deficiency list is created.

5.1.1.6 Output Testing

After performing the validation testing, the next step is output testing of the proposed system since no system could be useful if it does not produce the required output in a specific format. Asking the users about the format required by them tests the outputs generated or displayed by the system under consideration.

The output format of a screen is found to be correct as the format was designed in the system design phase according to the user needs. For the hard copy also, output comes out as the specified requirements by the user. Hence output testing does not result in any correction in the system.

5.2 TEST CASES

A specific set of steps and data along with expected results for a particular test objective. A test case should only test one limited subset of a feature of functionality. Test case documents for each functionality/ testing areas will be written, reviewed and maintained separately in Excel sheets.

In system testing, test data should cover the possible values of each parameter based on the requirements. Since testing every value is impractical, a few values should be chosen from each equivalence class. An equivalence class is a set of values that should all be treated the same. Ideally, test cases that check error conditions are written separately from the functional test cases and should have steps to verify the error message s and logs. Realistically, if error test cases are not yet written, it is OK for testers to check for error conditions when performing normal functional test cases. It should be clear which test data, if any, is expected to trigger errors.

CHAPTER 6
SYSTEM IMPLEMENTATION

6.1 INTRODUCTION

Implementation is the stage of project when the theoretical design is turned into a working system. At this stage, the main workload, the greatest upheaval and the major impact on existing practices shift to the user department. If the implementation stage is not carefully planned and controlled, it can cause chaos. The implementation stage is a system project in its own right. It involves careful planning, investigation of the current system and its constraints on implementation, design of methods to achieve the changeover, training of staff in the changeover procedure and evaluation of changeover methods.

The implementation is the final and important phase. It involves user training system testing in order to ensure successful running of the proposed system. Once the system design phase is over, the next stage is to implement and monitors the operation of the system to ensure that it continues the work effectively and efficiently.

The three main phases in implementation take place in series. These are the initial installation, the test of the system as a whole and evaluation maintenance and control of the system. The implementation plan and action to implement should be bound closely together. The implementation plan is a function of line management at least as far as key decision or alternative plans are concerned. The implementation plan was to convert the existing clerical files to the computer. The implementation plan listed all sub tasks so that individuals in the organization may be assigned specific responsibilities.

The installation of the new system that is bound to replace the current one may require a major revision of computer facilities as well as completely new after space. Space planning took into account the space occupied by the people, space by equipment and the movement of people and equipment in the working investment. After conduction the initial testing the system is loaded on the client office's computer. Some of the user employees in this case are selected. These users

are trained first and they run the system. A detailed documentation is prepared to this set of employees. There may be slight modifications to meet the organization.

After all modifications specified by the users in the documentation are made, the computer system is run along with manual system. Even though this kind of parallel run make extra burden to the employees and management, the system is run in parallel for the sake of checking reliability and efficiency.

After this document which compares the result of the manual system with those of the computerized is prepared. In the case of management many of whom participated in the development of the system short seminars were given. Particular attention was paid to the training of end users. The training sessions were aimed at giving the user staff the specific skills required in their new jobs. Education involved creating the right atmosphere and motivation of user staff. It explained the need for changes and helped to overcome the resentment caused by the feeling that computers took away the responsibility from individual departments.

Various measures have been taken by department officials in order to find suitable solutions by the following issues

- About the skill to be acquired.
- Reduction of man power in department.
- About the new form having all required option.

6.2 IMPLEMENTATION PROCEDURE

Implementation is the stage of the project where the theoretical design is turned into a working system. At this stage, the main work load, the greatest upheaval and the major impact on the existing system shifts to the department. If the implementation is not carefully planned and controlled, it can cause confusion. Implementation includes all those activities that take place to convert from the old system to the new one. Proper implementation is essential to provide a reliable system to meet the organizational requirements.

Successful implementation may guarantee improvement in the organization using the new system, but improper installation will prevent it. The process of putting the developed system into the actual use is called system implementation. This includes all those activities that take place to convert from the old system to the new system. The system can be implemented only after through testing is done and if it is found to be working according to the specification of the system.

The most crucial stage is achieving a new successful system and giving confidence on the new system for the user that it will work efficiently. It involves careful planning, investigation of the current system and is constraints on implementation, design of methods to achieve the changeover. The more the complex system being implemented is, the more involved will be the system analysis and the design effort required for its implementation.

6.2.1 EQUIPMENT ACQUISITION

Here, the necessary equipment is acquired to implement the system. Major steps involved in the implementation are installation of software. In case of this system there is no special hardware requirement for the working of the software. In addition to basic hardware requirements one thing that is essential is the software requirements with an operating system that has server support. The proposed system can run on any PC, which works on android and any versions of it. Our software is platform dependent and will run only on the windows platform after installing the software, it is essential to ensure that the software is working accordingly with the existing software.

6.2.2 TRAINING

For this system, it was explained to the users how to use the system, what details are to be given while creating a new profile, how to use it so that we get the maximum output out of it. The proposed system may be entirely new, replacing an existing one or it may be modifications to the existing system. In either case, proper implementation is necessary to provide a reliable system to meet organizational requirements.

6.2.3 EVALUATION

Evaluation of the system is performed to identify its strength and weaknesses.

6.3.2.1 OPERATIONAL EVALUATION

Assessment of the manner in which the system functions, including ease of use, response time, overall reliability and level of utilization.

6.2.4 ORGANIZATIONAL IMPACT

Identification and measurement of benefits to the organization in such areas like financial concerns, operational efficiency and competitive impact.

6.2.5 USER MANAGEMENT ASSESSMENT

Evaluation of attitude of senior and user managers within the organization, as well as end users.

6.2.6 DEVELOPMENT PERFORMANCE

Evaluation of the development process based on overall development time and effort, conformance to budgets and standards, and other project management criteria includes assessment of development methods and tools.

6.2.7 DOCUMENTATION

After the testing and implementation was completed, the whole system was presented and documented in a readable manner. This was done to ensure that any corrections, manipulations or updating are performed in future, the users would face no problem in performing those changes. Documentations include the source code, the tables that were used to construct the base for the system, the framework which is bound to the programs etc.

CHAPTER 7

SOFTWARE MAINTENANCE

Software maintenance denoted any changes made to a software product after it has been delivered to the customer. Maintenance is inevitable for almost any kind of product. Most products need maintenance due to wear and tear by use. Although software does not wear out like a piece of hardware it “ages” and eventually fails to perform. So maintenance becomes necessity.

Types of software maintenance:

7.1 CORRECTIVE MAINTENANCE

Corrective maintenance of a software product is necessary either to rectify the bugs observed while the system is in use.

7.2 ADAPTIVE MAINTENANCE

A software product might need maintenance when the customers need the product to run on new platforms, on new operating systems, or when they need the product to interface with new hardware or software.

7.3 PERFECTIVE MAINTENANCE

A software product needs maintenance to support the new features that users want it to support, to change different functionalities of the system according to customer demands, or to enhance the performance of the system.

7.4 PREVENTIVE MAINTENANCE

Modification of a software product after delivery to detect and correct latent faults in the software product before they become effective faults.

CHAPTER 8

CONCLUSION

The project was successfully completed within the time span allotted. Every effort has been made to present the system in a more –user friendly manner. All the activities provide a feeling like an easy walk over to the user who is interfacing with the system. The system has been developed with much care and free of errors and at the same time it is efficient and less time consuming. The purpose of this project was to develop a web application for an e-commerce platform, enabling buyers and sellers to connect and transact seamlessly.

This project helped us in gaining valuable information and practical knowledge on
several topics likes deigning web pages using Python and HTML, usage of responsive templates and customized user forms. The project is created using MySQL as back end which also helped to study about the various operations associated with MySQL. The system as a whole is secured. Also the project helped us in understanding about the development phases of a project and software development life cycle. This project has given greater satisfaction because we developed a platform that helps to promote commerce, connect buyers and sellers, and foster economic growth.

CHAPTER 9

FUTURE ENHANCEMENT

Future Enhancement for the workers platforms:

1. Integration with Emerging Technologies like AI-powered chatbots, augmented reality, or blockchain to enhance user experience and security.

2.Expanded Service Offering: Add more service categories, such as specialized skilled trades or ncreative services, to attract a broader user base.

3. Enhanced Analytics and insights: Provide workers and admins with detailed analytics and insights to help them optimize their services and business strategies.

4.Mobile App Features: Introduce features like in-app messaging, GPS tracking, and push notification to improve user engagement and worker efficiency.

5.Personalized Recommendations: Develop a system that offers personalized worker recommendations based on user preferences, behaviour, and job requirements.

6. Multi-Language Support: Add support for multiple languages to cater to a diverse user base and expand the platform's reach.

7. In-App Payment Options: Integrate various payment options, such as digital wallets or cryptocurrencies, to provide users with more flexibility and convenience.

CHAPTER 10

APPENDIX

10.1 SCREENSHOTS

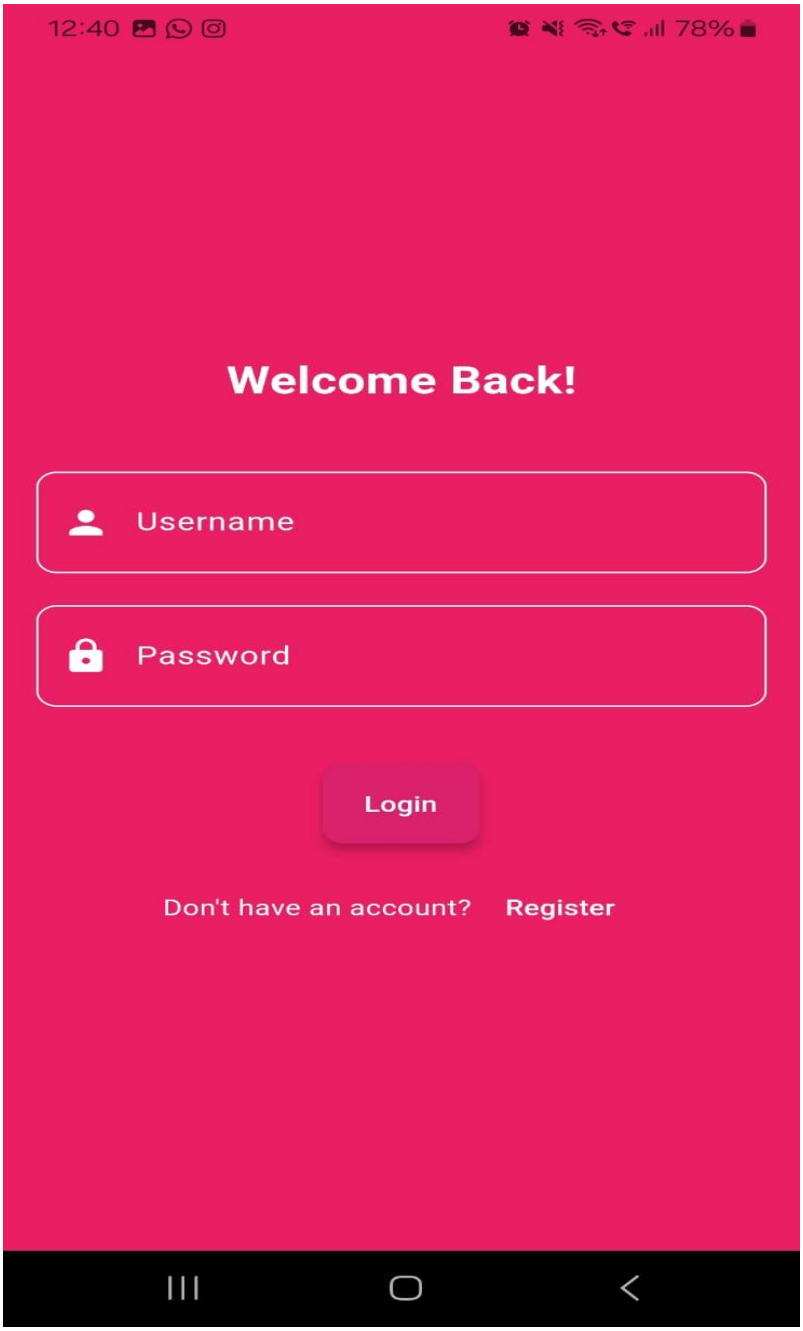


Fig: 10.1.1 login

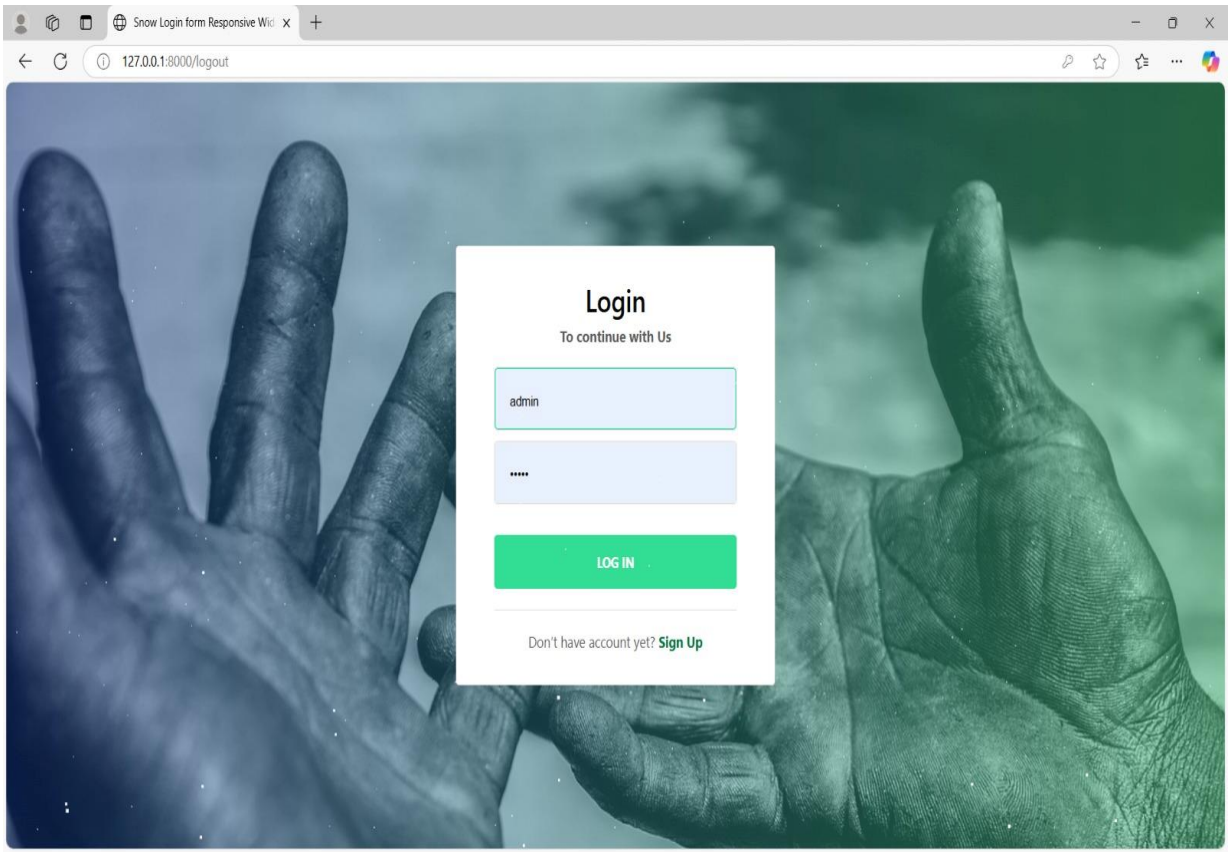


Fig: 10.1.2 Login Page

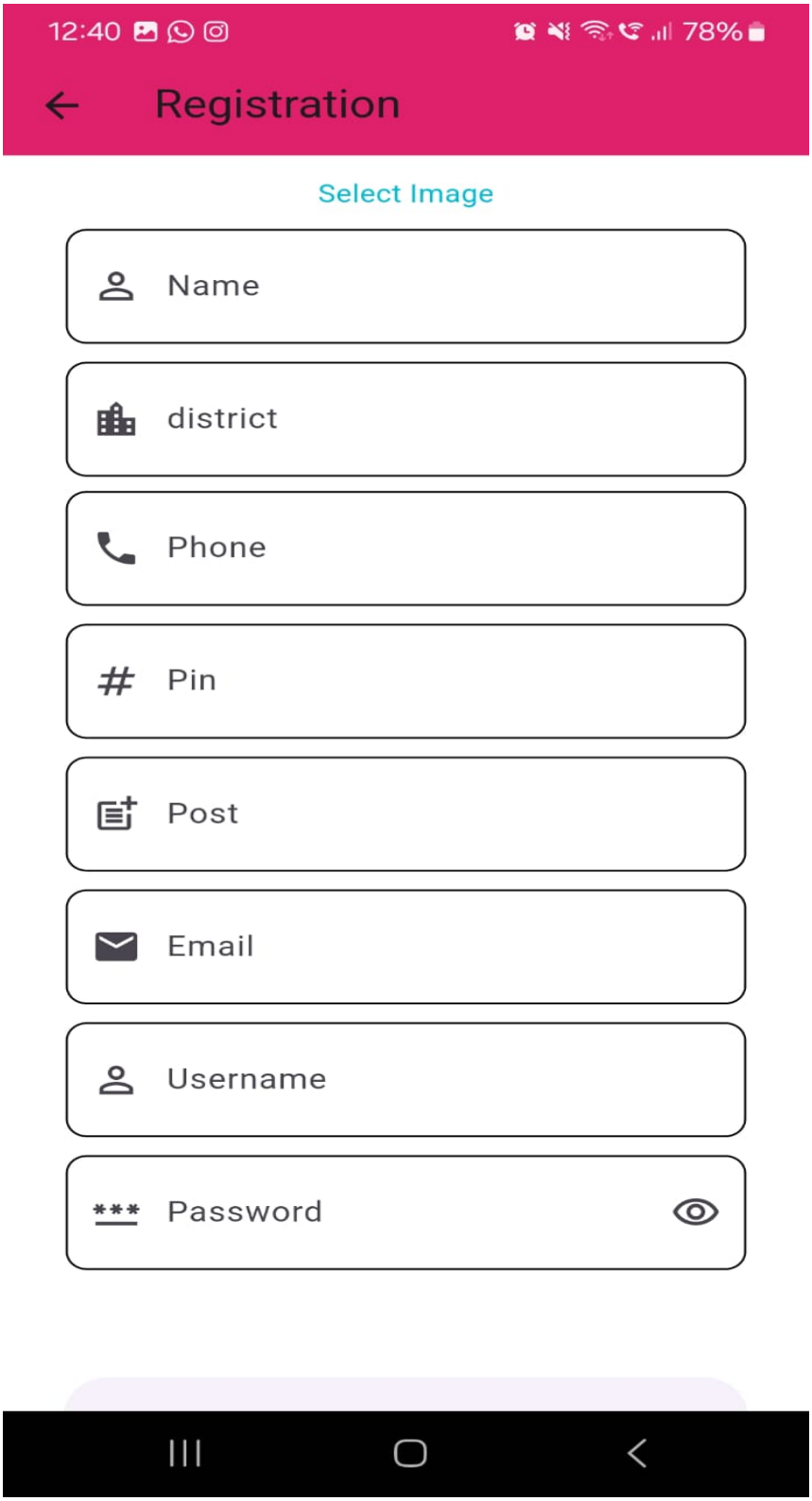


Fig: 10.1.3 Registration

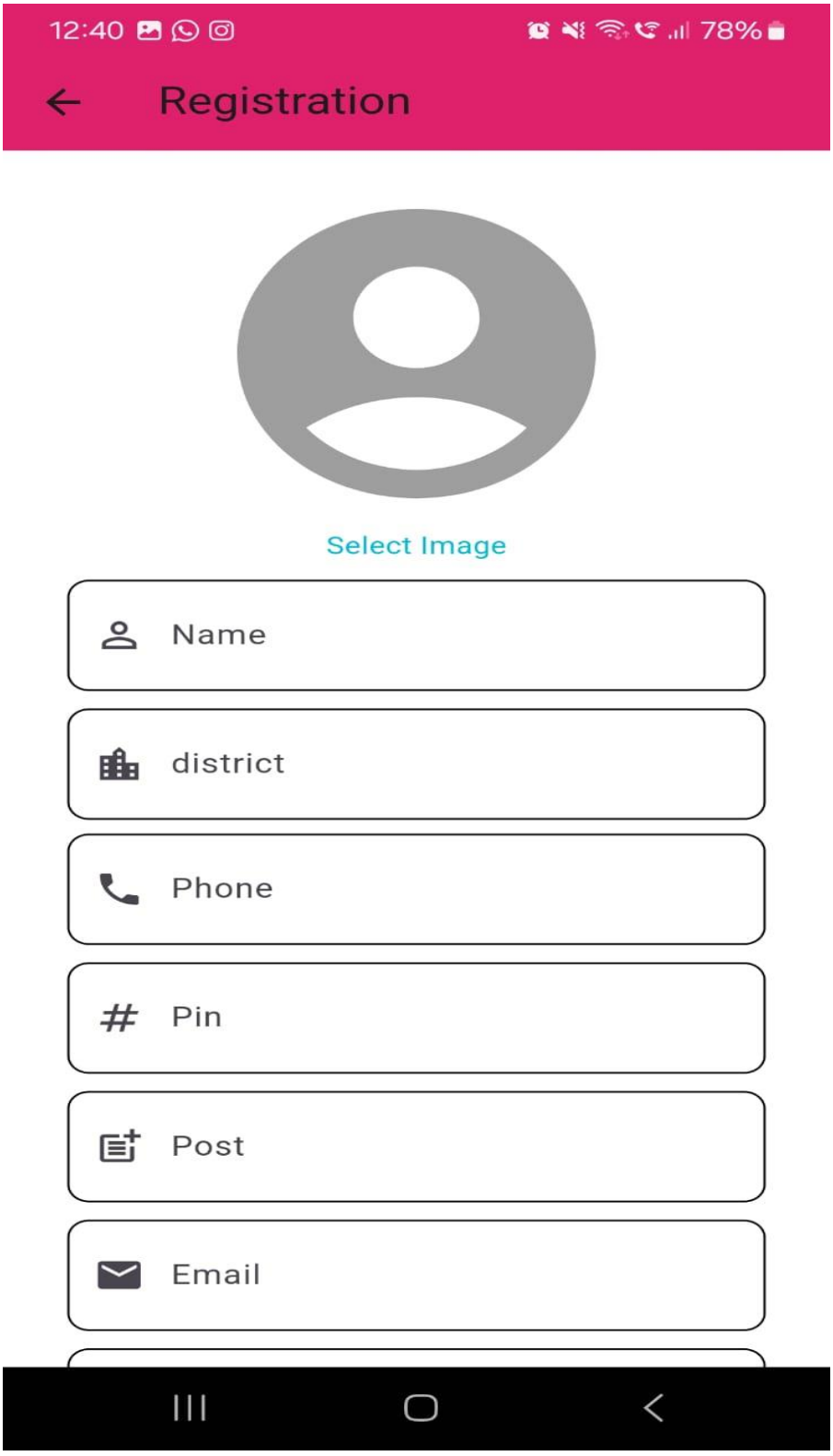


Fig: 10.1.4 Registration

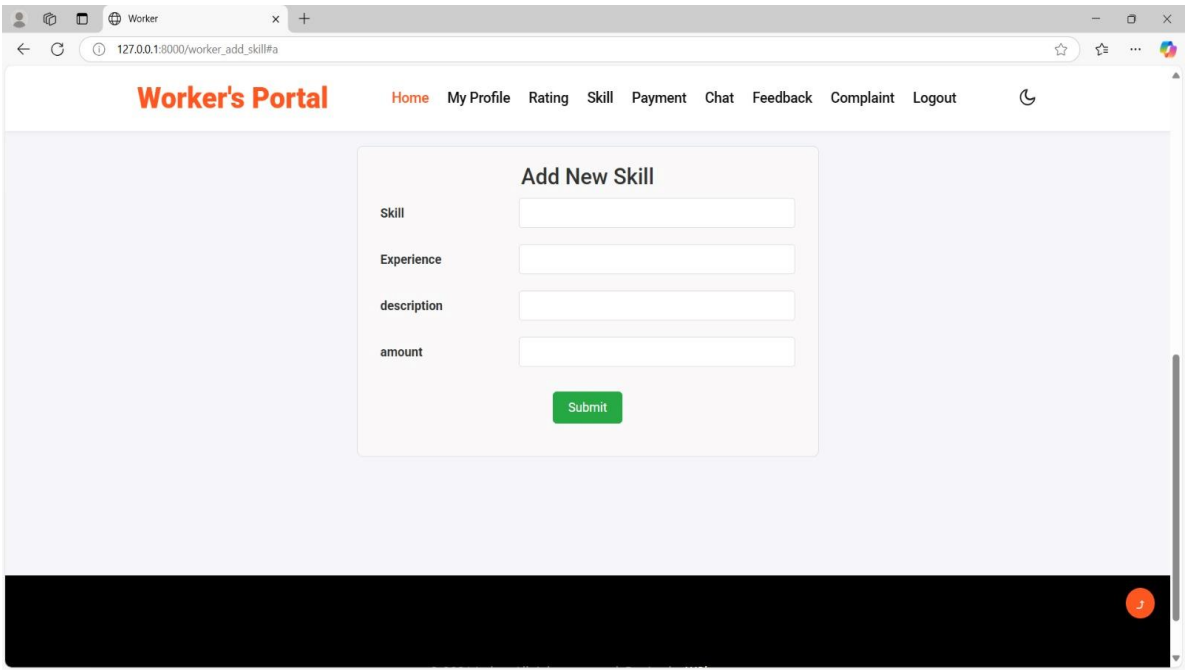


Fig: 10.1.5 Change password

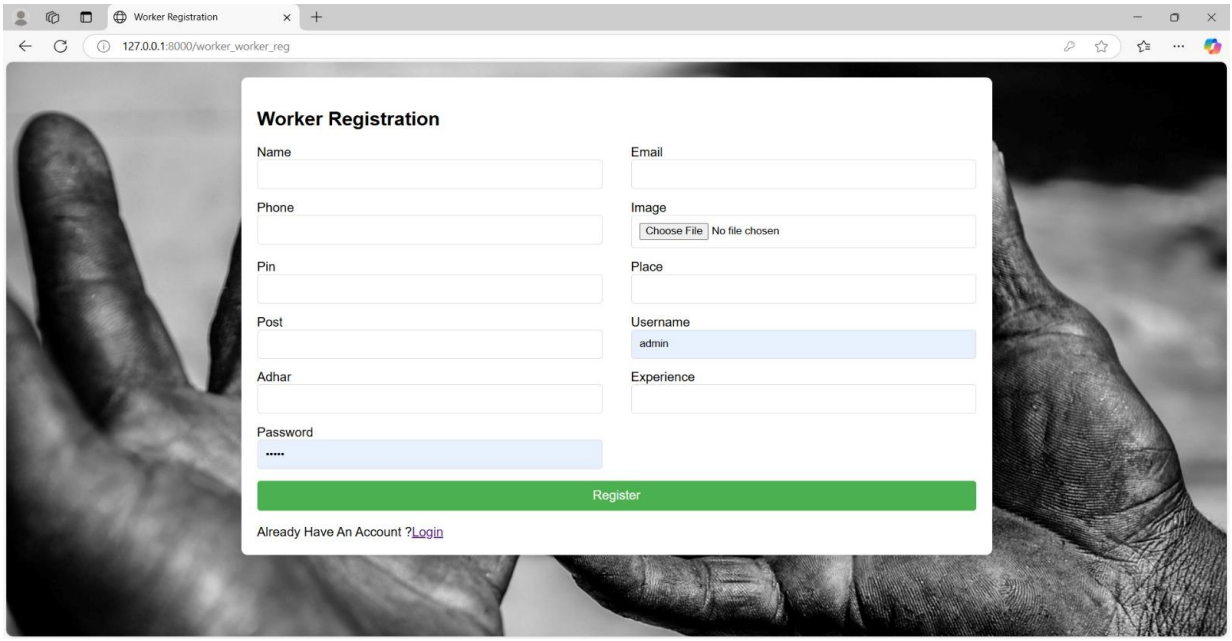


Fig:10… Worker Registration

10.2 Sample source code

```
Django settings for workersportal project.

Generated by 'django-admin startproject' using Django 3.2.24.

For more information on this file, see
https://docs.djangoproject.com/en/3.2/topics/settings/

For the full list of settings and their values, see
https://docs.djangoproject.com/en/3.2/ref/settings/
"""
import os
from pathlib import Path

# Build paths inside the project like this: BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent

# Quick-start development settings - unsuitable for production
# See https://docs.djangoproject.com/en/3.2/howto/deployment/checklist/

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = 'django-insecure-%0zbal=2b4#=l9^+@^4p&-2-
t$rv%c&l&rmhm^mlxba4ria$ar'

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = True

ALLOWED_HOSTS = ['*']

# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
```

```
    'myapp.apps.MyappConfig',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'workersportal.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [os.path.join(BASE_DIR, 'templates')]
        ,
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

WSGI_APPLICATION = 'workersportal.wsgi.application'

# Database
# https://docs.djangoproject.com/en/3.2/ref/settings/#databases

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
```

```
        'NAME': 'workershub',
        'USER': 'root',
        'PORT':3306,
        'HOST': 'localhost',
        # 'PASSWORD':'123456789'
    }
}

# Password validation
# https://docs.djangoproject.com/en/3.2/ref/settings/#auth-password-validators

AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]

# Internationalization
# https://docs.djangoproject.com/en/3.2/topics/i18n/

LANGUAGE_CODE = 'en-us'

TIME_ZONE = 'UTC'

USE_I18N = True

USE_L10N = True

USE_TZ = True
```

Static files (CSS, JavaScript, Images)
https://docs.djangoproject.com/en/3.2/howto/static-files/

STATIC_URL = '/static/'
STATICFILES_DIRS=[os.path.join(BASE_DIR,'static')]
MEDIA_URL = 'media/'
MEDIA_ROOT=os.path.join(BASE_DIR,'media')

Default primary key field type
https://docs.djangoproject.com/en/3.2/ref/settings/#default-auto-field

DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'

*DATA_UPLOAD_MAX_MEMORY_SIZE=1024*100000000000000*

t.com/en/3.2/howto/static-files/

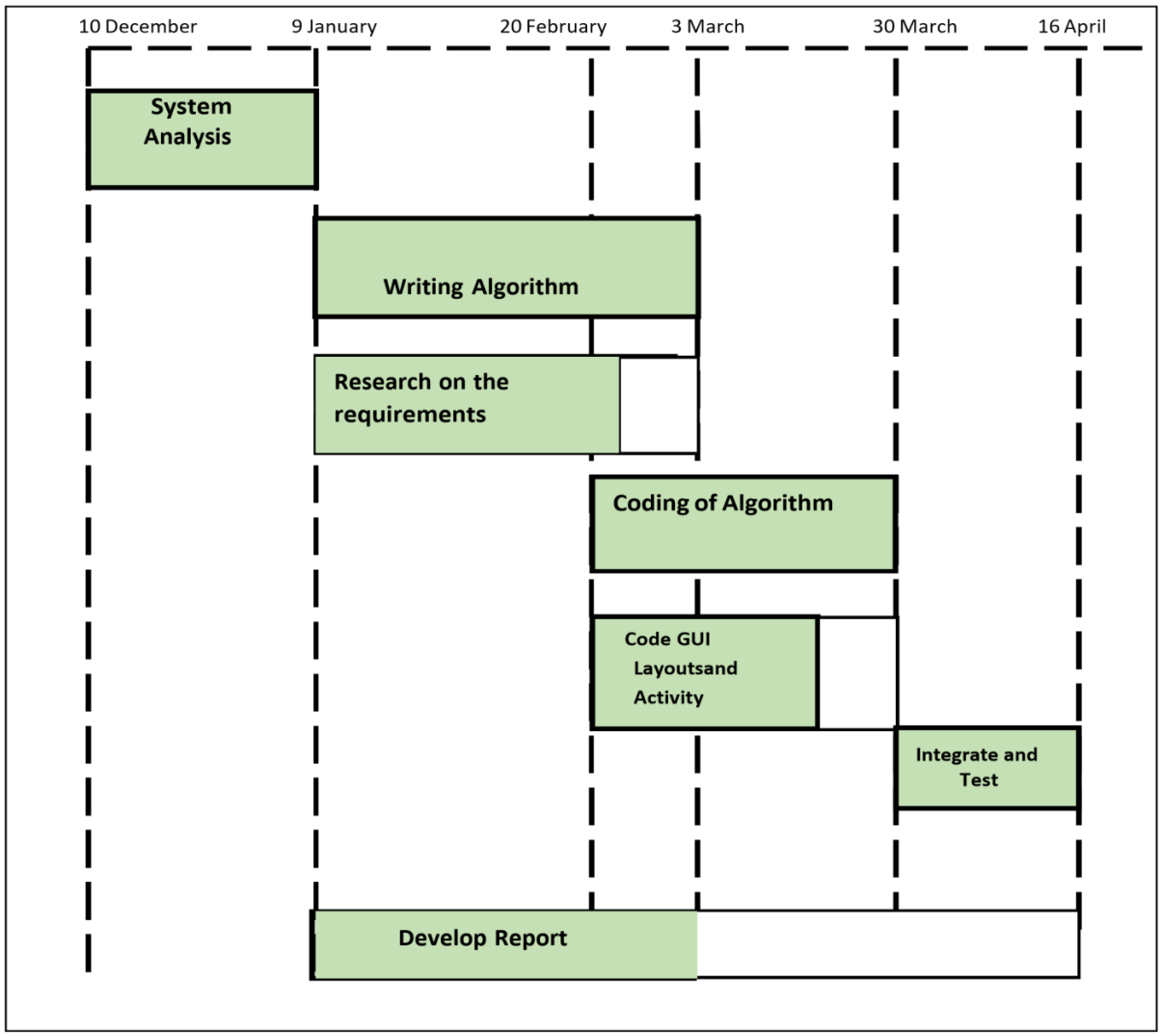
STATIC_URL = '/static/'
MEDIA_ROOT = os.path.join(BASE_DIR,'media')
MEDIA_URL='/media/'

Default primary key field type
https://docs.djangoproject.com/en/3.2/ref/settings/#default-auto-field

DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'

10.2 GANTT CHART

The Gantt chart was developed by Henry Gantt. Those are used in software project management and enhanced version of standard Gantt chart. These are mainly allocating resources to activities. It is a special type of bar chart. Each bar represents an activity. Bars are drawn alone a time line. Length of each bar is proportional to duration of time planned for corresponding activity.



CHAPTER 11
REFERENCE

Text Books:

- **Fundamentals of Software Engineering, Fifth Edition, Rajib Mall- PHI 2018**
- **Software Engineering: A Practitioner's Approach (IRWIN COMPUTER SCIENCE) Hardcover – 16 March 2014**

Websites:

- **https://www.tutorialspoint.com/uml/uml_class_diagram.html**
- **https://www.w3schools.com/python/**
- **pygame (github.com)**
- **https://myiee.org/documentation-requirements**
- **https://www.w3schools.com/mysql/default.asp**